

Streaming Data Management and Time Series Analysis Project

University of Milano-Bicocca

Muhammad Qasim - 897683 (m.qasim1@campus.unimib.it)
A.A 2023/2024

Overview

Time series data is a **sequence of data points** that are collected at intervals over a period of time. What makes a time series dataset unique is the sequence or order in which these data points occur. This ordering is vital to understanding any **trends, patterns, or seasonal variations** that may be present in the data.

Time series forecasting employs specialized statistical techniques to effectively model and generate future predictions. It is commonly used in **business, finance, environmental science**, and many other areas for **decision-making and strategic planning**.

In this project, we'll start by understanding what differentiates time series from static data and will go through a step-wise implementation of forecasting.

Project Objective

Aim is to **develop and evaluate predictive models** for forecasting the average number of days needed to close requests. The **dataset** provided spans from **January 4, 2007, to March 31, 2015**, and includes three fields:

- **date**: a string representing the date in the format yyyy-mm-dd
- **weekday**: a string representing the name of the weekday
- **ave_days**: a floating point number representing the average number of days needed to close the requests that were closed on that day.

The **goal** is to produce **forecasts** for each day from **April 1, 2015, through November 7, 2015**. Specifically, three sequences of forecasts must be generated using different **modeling approaches**:

- **ARIMA** (AutoRegressive Integrated Moving Average) model
- **UCM** (Unobserved Components Model)
- **ML** (Machine Learning) model

The forecasts will be **evaluated** based on the **Mean Absolute Error (MAE)**, and a ranking of the forecasts will be periodically updated.

Pre-processing

The dataset consists of two **columns**: the **date**, which runs from January 4, 2007, to March 31, 2017, in the **format dd/mm/yyyy**; and the columns **weekday** and **ave_days**. The dataset underwent several preprocessing steps to ensure its suitability for analysis.

```
df.shape, df.columns, df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3009 entries, 0 to 3008
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   date        3009 non-null   object 
1   weekday     3009 non-null   object 
2   ave_days    2807 non-null   float64
dtypes: float64(1), object(2)
memory usage: 70.6+ KB
((3009, 3), Index(['date', 'weekday', 'ave_days'])
```

First, the 'date' column was converted to a **datetime format** to enable time-based operations. A check for duplicates confirmed that there were no duplicate records in the dataset.

An examination for missing values revealed that the 'ave_days' column had **202** missing entries. To address this, the missing values were imputed by calculating the **weekly mean** of 'ave_days' and filling the missing values with these weekly averages. This was achieved by extracting the week number from the 'date' column using the **ISO calendar week** method. After imputing the missing values, the temporary 'week' column was removed from the dataset. These preprocessing steps ensured that the dataset was clean, complete, and ready for further analysis.

	weekday	ave_days
date		
2007-01-04	Thursday	0.179487
2007-01-05	Friday	0.500000
2007-01-06	Saturday	57.539966
2007-01-07	Sunday	57.539966
2007-01-08	Monday	3.521739
...	...	...
2015-03-27	Friday	38.580756
2015-03-28	Saturday	1.040000
2015-03-29	Sunday	42.000000
2015-03-30	Monday	63.477124
2015-03-31	Tuesday	40.496689

3009 rows × 2 columns

Exploratory and Series Analysis

The ave_days dataset exhibits substantial variability with a symmetric distribution around the central values (mean and median). While most values cluster around the mean, the presence of a wide range and high standard deviation indicates significant outliers and variability. These summary statistics are crucial for understanding the data's overall behavior and for informing further statistical analysis or modeling efforts.

(3009, 9)		
	statistic	value
0	Minimum	0.000000
1	1st Quartile	15.068966
2	Median	31.218638
3	Mean	31.782398
4	3rd Quartile	44.665615
5	Maximum	97.186992

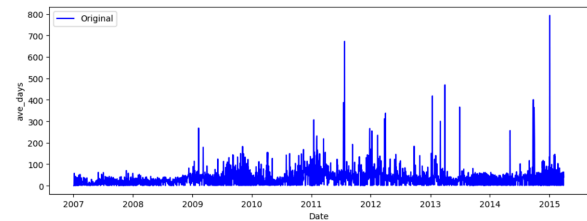
Time series data can exhibit a variety of patterns, and it is often helpful to split a time series into several components, each representing an underlying pattern category.

Time Series Decomposition

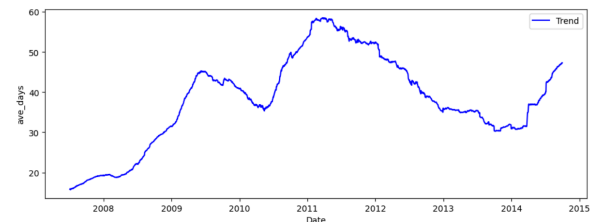
By using **time series decomposition**, we split the data into four main components:

The first graph shows the original time series as it exists normally. The value

(ave_days) is shown for each day of the year at the level it was at on each day.



Trend: Trend represents a long-term increase or decrease in the time series.

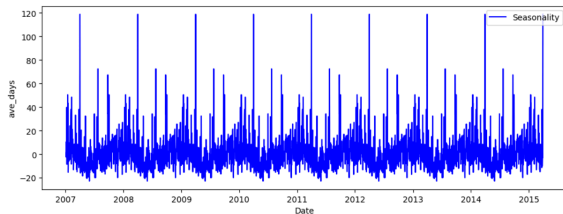


In the figure above we can see that the trend in the ave_days value over the period of **2007 to 2015**. The trend displays there are actually two main **different trends**. There are years up until about **2011** where **ave_days tends to increase** considerably and years **after 2011 drastically decreases** and almost remains as **low** as when the testing of **ave_days began in 2015** with just a slight increase over time.

Cycles: In simple words, datasets that contain seasonality with no fixed frequency are cyclic data.

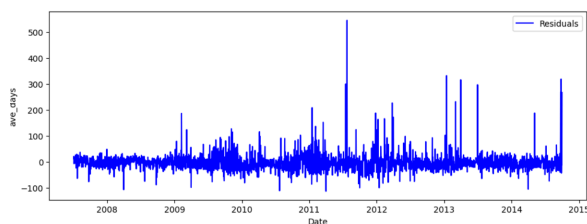
When we decompose a time series into components, we usually combine the trend and cycle into a single **trend-cycle** component (sometimes called the **trend** for simplicity).

Seasonality: Seasonality shows a uniform repeating pattern at a constant frequency in the time series.



The graph above shows seasonality of the ave_days over the period of 2007 to 2015. The ave_days tend to be lower near the beginning of each year and higher in the middle of the year. The seasons of the year have differing levels of ave_days but remain pretty consistent from year to year.

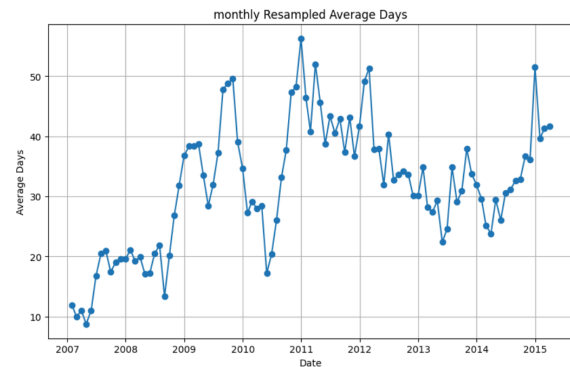
Residual: The data remaining after removing the above three (Trend-Cycles , Seasonality,) is called Residual.



The graph shows there are some very irregular ave_days points around the middle of the year 2011, but the ave_days irregularities seem to be pretty consistent.

To gain deeper insights into the underlying trends and patterns within the time series data, we employed a **resampling and aggregation technique**. This technique

allows us to transform the data into a more manageable form while preserving its temporal characteristics.



The resulting resampled and aggregated dataset provides valuable insights into the activity level on a monthly basis, allowing us to identify long-term trends, seasonality, and other temporal patterns that may exist within the data.

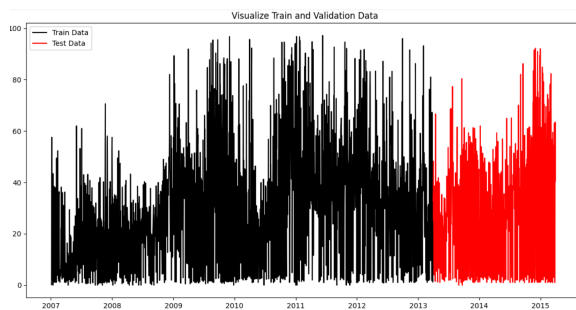
ARIMA

A wide array of methods are available for time series forecasting. One of the most commonly used is **Autoregressive Moving Average (ARMA)**, which is a statistical model that predicts future values using past values.

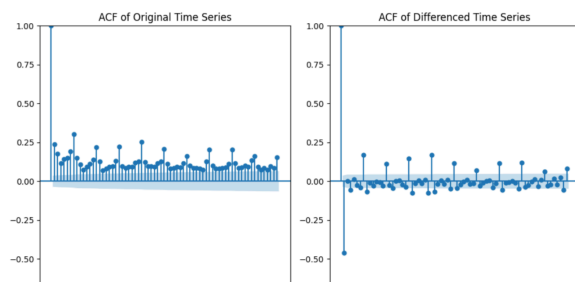
An extension of ARMA is the **Autoregressive Integrated Moving Average (ARIMA)** model, which doesn't assume stationarity but does still assume that the data exhibits little to no seasonality. Fortunately, the seasonal ARIMA (SARIMA) variant is a statistical model that can work with non-stationary data and capture some seasonality.

Python provides many easy-to-use libraries and tools for performing time series forecasting in Python. Specifically, the **stats library** in Python has tools for building ARMA models, ARIMA models and SARIMA models with just a few lines of code.

Next we **split data** into two parts: the **Train Set**, which runs from **2007-01-04** until **2013-03-30**, and the **Test Set**, which runs from **2013-03-31** until **2015-03-31**.



Our approach to tackle this forecast. First, we identified seasonality through **ACF/PACF plots**, and then we explored various ARIMA configurations.

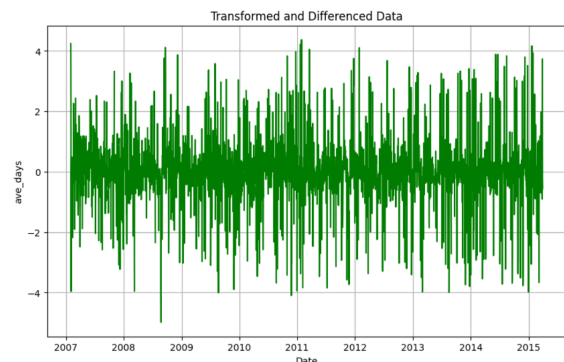


Non-Stationary time series data can still be used to build forecasting models. They

would require additional data pre-processing steps to remove the trend or seasonality present in it.

Applied common methods to convert a **non-stationary time series into a stationary one**:

1. **Differencing**: This method involves subtracting the value of the time series at a previous point in time from the current time series value, in order to remove trends in the data.
2. **Log transformation**: This method involves taking the logarithm of the values of the time series, this removes the exponential growth present in the data.



We initially applied the **extracted seasonal parameter**, followed by experimenting with **random p and q** values. Additionally, we applied **grid search** for **hyper-parameter optimization**. By comparing each model's performance, we evaluated **diagnostic results** like **residuals**, **goodness-of-fit**, and **MAE (Mean Absolute Error)**. This approach ensured we selected the ARIMA model that effectively captured the time series

characteristics and delivered the most accurate forecasts.

The Parameters we examined are as follows:

- **ARIMA(0, 1, 0)(0, 1, 0)[24]**

The resulting model had an **AIC** of **9683** and an **MAE** of **45**.

This, together with the visual inspection of the Normal Q-Q plot, which shows a lack of normality in the residuals, indicates that this parameterization is not ideal. As a result, further investigation of various p and q values is required to select a model that better represents the underlying structure of the data and gives more accurate results.

- **ARIMA(0, 1, 0)(0, 1, 0)[7]**

While this model outperforms the prior attempt with an **AIC** of **8184** and an **MAE** of **34**, some significant difficulties remain.

Despite the decreased **AIC** and **MAE**, normality in the residuals remains lacking, as evidenced by the lack of agreement between the data and the normal distribution in the Q-Q plot. Additionally, the residuals over time plot suggest seasonality, which is verified by the correlogram plot. These considerations show that the model requires further modification to produce more accurate and statistically sound findings.

- **ARIMA(0, 1, 0)(1, 1, 1)[24]**

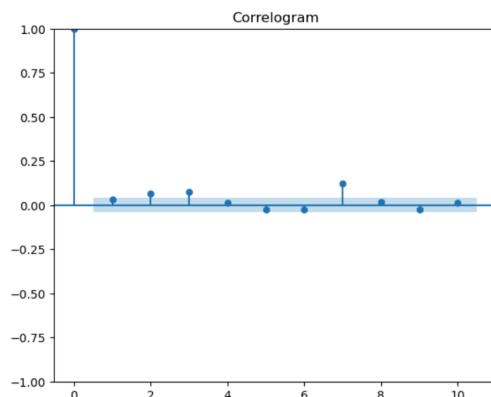
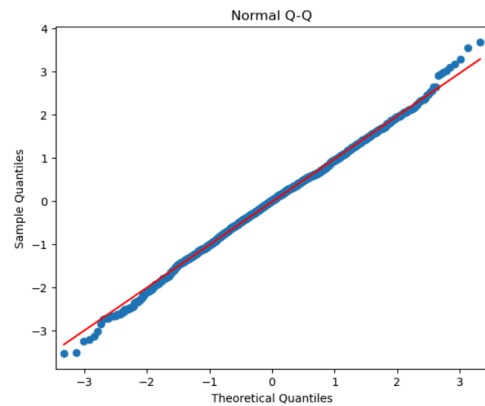
This model further reduces the **AIC** score (**8180**) and **MAE** (**2**) compared to the previous option, it still requires refinements to address persistent issues.

Similar to the **ARIMA(0, 1, 0)(0, 1, 0)[7]** model, it has non-normal residuals as shown in the Q-Q plot, as well as probable seasonality as shown by the residuals over time plot and validated by the correlogram. The decreasing error metrics indicate promise, but addressing normalcy and seasonality difficulties is critical to generating statistically valid and reliable results.

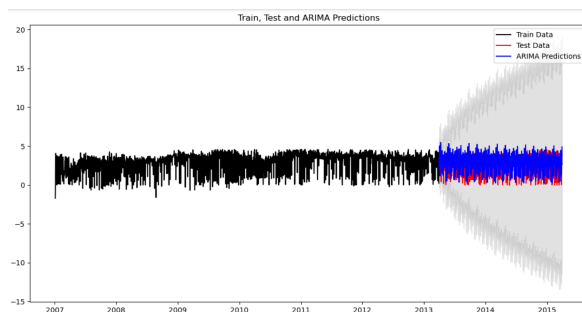
- **ARIMA(6, 1, 6)(0, 1, 0)[24]**

Proceeding with grid-search, this model has a substantially lower **AIC** (**7399**) and **MAE** (**1.17**) than prior attempts, indicating better fit and accuracy.

Furthermore, visual inspection of the **Q-Q plot** indicates that the residuals are normal, which is a critical requirement for a more sound statistical measurement. The lack of seasonality trends in the residuals plot and **correlogram** supports this model's appropriateness.

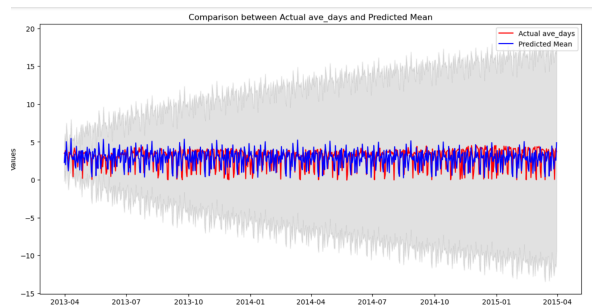


While additional testing is recommended, the **ARIMA(6, 1, 6)(0, 1, 0)[24]** model appears to be the best option based on the current findings from the AIC, MAE, and diagnostic plots. This model stands out as the current top performer for accurate and statistically reliable forecasting.



A comparison between the original series and the predictions provided by the

improved Arima **ARIMA(6, 1, 6)(0, 1, 0)[24]** can be seen in the below figure.



During our exploration, we found additional ARIMA models with lower AIC and MAE scores (**p=6, q=6**). However, they failed in essential key areas. Comparing these models to the **ARIMA(6, 1, 6)(0, 1, 0)[24]** model, the latter showed more accuracy on the test set and also in diagnostic tests. This emphasizes how critical it is to prioritize overall model performance above and beyond measurements such as AIC and MAE since real-world accuracy and diagnostic evaluations are essential for choosing a strong and trustworthy forecasting model.

UCM

For a better interpretation of seasonal time series, other methods have been developed such as the **Unobserved Components Model (UCM)**. Being a so-called 'state space' model, UCM decomposes the original time series to its individual **level, trend, cyclic, seasonal components** and predicts future values by modeling and taking the sum of these components.

Interestingly, UCMs bridge the gap between the flexibility of ARIMA models and the interpretability of smoothing models.

It is important to note that the train-test split was not modified for this iteration. Maintaining the same **80/20** split.

As for the UCM part, different models were tested again to obtain the one that best fit the data. Thus, the level and seasonal parameters were considered.

The **level parameter** that models the trend component, was initially considered. Considering daily seasonality, the most evident type of seasonality within the available data, different types of models were tested by varying the level parameter between the following values: **No trend, Local level, Random walk, Deterministic trend, Local linear deterministic trend, Random walk with drift, Local linear trend, Smooth trend**. Taking MAPE and secondarily AIC as metrics, the type of level that yielded the best results was **rwdrift**, which is a random walk with drift.

It is possible to observe the results obtained in Table below.

Level	AIC	MAPE
llevel	8973.66	0.46
rwdrift	9343.82	0.43
ntrend	11832.11	1.47
Rwalk	9350.27	0.51
lldtrend	8971.04	0.38
lltrend	10044.68	57.77
strend	10042.50	57.78
rtrend	10287.24	98.34

Table: Modeling Component of UCM models

Next regarding the seasonal component, making several attempts, it was decided to opt for 3 sine waves of period 24 and 2 sine

waves of period 168 to model daily seasonality.

Thus, considering the final model, the best results were obtained with a level equal to random walk with drift and the seasonal part modeled by 3 sine waves of period 24 and 2 sine waves of period 168.

The trained model recorded on the test set a **MAPE** value of **0.4903**. The result from the graphical point of view of fitting the model on the testing data can be observed next figure

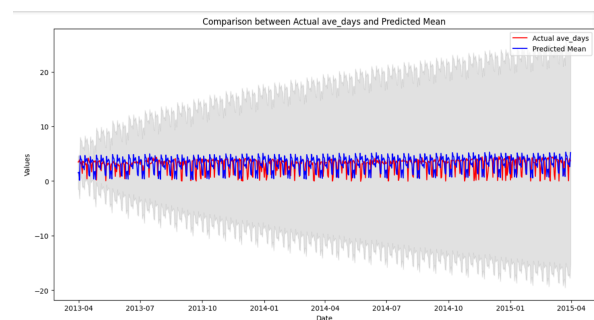


Figure: Prediction on the UCM test set

Machine Learning

The machine learning process involved several key steps, beginning with the definition of the predictive problem at hand. Our objective was to develop models capable of predicting the 'ave_days' variable based on a set of **features extracted** from the dataset, including temporal attributes such as **dayofweek, quarter, dayofmonth, month, year, Day_of_year, Weekend, and season**.

Following the problem definition, the dataset underwent preprocessing to clean and prepare the data for modeling. This included extracting additional features from the 'date' attribute to enrich the dataset

with temporal information. Subsequently, the dataset was split into training and test sets, with **2156** instances allocated to the training set and **539** instances to the test set.

We then selected and evaluated three different **machine-learning models**: **Random Forest** , **Decision Tree**, and **SVR**.

Each model was trained using the training dataset and evaluated using evaluation **metrics such as: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE)** on the test dataset.

The results of the evaluation revealed that the **Random Forest model outperformed** the other models, demonstrating **lower error metrics** and **higher predictive accuracy**. This suggests that the Random Forest model is the most suitable for predicting the '*ave_days*' variable in our dataset.

Overall, the machine learning process involved iterative steps of data preprocessing, model selection, training, and evaluation to develop and assess predictive models capable of accurately predicting the target variable.

SVR

Support Vector Regression (SVR) is a machine learning algorithm used for regression tasks. It works by finding the optimal hyperplane that best fits the

training data while minimizing prediction errors.

SVR is particularly useful for datasets with complex relationships and non-linear patterns, as it can capture these intricacies effectively. However, SVR may be sensitive to its hyperparameters and may require careful tuning to achieve optimal performance.

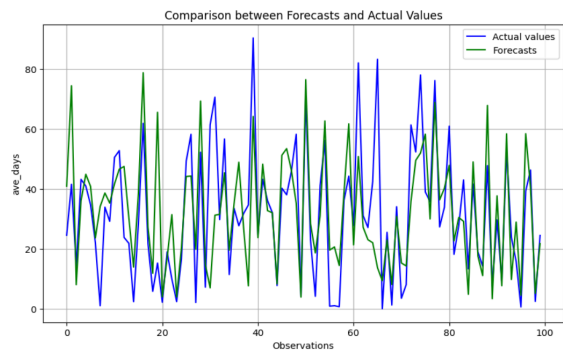
Applying this type of model after standardizing the input data, predictions were made on the validation set returning an MAE of **24.17**.

The performance metrics obtained from the evaluation of the SVR model indicate that it struggles to accurately predict the target variable compared to the other models evaluated. The relatively high RMSE, MAE, and MAPE values suggest that SVR may not effectively capture the underlying patterns and relationships present in the dataset. This could be due to challenges in finding the optimal hyperparameters for the SVR model or limitations in its ability to handle the complexity of the data.

Random Forest

The Random Forest model played a pivotal role in our predictive analysis, offering valuable insights and accurate forecasts for the '*ave_days*' variable. Here's a detailed examination and commentary on the relevance and performance of the Random Forest model.

Random Forest emerged as a compelling choice for our predictive modeling task due to its ensemble learning nature and robustness to overfitting. In our dataset, which comprises various temporal attributes and features extracted from the date attribute, Random Forest's ability to handle non-linear relationships and capture complex interactions proved highly relevant.



In particular, when applying Grid Search to Random Forest, an MAE of 11.03 is achieved on the validation set.

These algorithms, including Random Forest, are well-suited for consumption forecasting due to their ability to model intricate relationships among multiple variables and effectively handle datasets with numerous variables or heterogeneous data.

Decision Tree

Decision Trees are well-known for their simplicity and interpretability, making them an attractive choice for tasks where understanding the decision-making process is important. However, Decision Trees can be susceptible to overfitting and may struggle with capturing complex, non-linear

relationships present in the data. In this case, the DecisionTree achieves an **MAE of 21.04**, which suggests decent performance but falls short of the Random Forest model. While Decision Trees provide valuable insights into feature importance and decision rules, their limitations in handling complex relationships might hinder their predictive power compared to other models like Random Forests. It's important to consider these trade-offs when choosing the appropriate model for your specific needs.

This table summarizes the performance of the three regression models based on their evaluation metrics.

Model	Root Mean Squared Error	Mean Absolute Error	Mean Absolute Percentage Error
RandomForestRegressor	43.37	17.95	2.41%
DecisionTreeRegressor	52.53	21.04	2.42%
SVR	48.22	24.18	3.84%

Conclusion:

In summary, the **RandomForestRegressor** showed the best performance among machine learning models with an **RMSE of 43.37**, **MAE of 17.95**, and **MAPE of 2.41%**. The **ARIMA** model achieved a strong balance between accuracy and simplicity with an **AIC of 7399** and **MAE of 1.17**. The **UCM** provided the most detailed decomposition and interpretability, with competitive accuracy, achieving a **MAPE of 0.43** and an **AIC of 9343.82**. Each model has its strengths, making them suitable for different aspects of time series forecasting depending on the specific requirements of accuracy, interpretability, and component analysis.